Test Report: CART-Frontend_AddToCart

Jaiganesh — Add-to-Cart Integration & Guest Resume Flow (Module 6: Cart, Team GALXY)

Environment Note

This review was performed in a sandboxed environment with no network access. **npm install** / **next build** / **next dev** could not be executed (npm registry calls returned 403 Forbidden). The assessment below is a thorough static and logical code review against the assignment brief (*Jaiganesh_Frontend_AddToCart_Assignment.pdf*), tracing the actual request/response flow through the mock API routes and React components.

Requirements Coverage

Requirement	Status	Notes
Add to Cart → POST /api/cart/items with product_id, selected_attributes, quantity	**PASS**	Payload correctly assembled in handleAddToCart; also includes custom_text and ai_preview_image
ai_preview_image hand-off from AI generation step	**PASS**	aiPreviewImage state flows into payload; mock generateAIPreview() simulates Module 5
Guest → signup/login → auto-add resume flow	**PASS**	Unauth users: payload saved to sessionStorage (pending_cart_item), redirected to /login; AuthContext auto-fires POST once user is set
Resume flow is purely client-side, no unauth API call	**PASS**	Confirmed - POST only fires after login succeeds
201 success → toast + badge count update	**PASS**	triggerToast(success) + refreshCart() updates cartCount in Header
400 invalid-attribute → inline errors	**PASS**	Mock API returns errors: {font, color}; mapped into inlineErrors and passed to AttributeRenderer
404 product unavailable	**PASS**	Dev-only toggle simulates trigger_404_inactive; handled with generalError + toast
Stale-item UI states (is_available, needs_attention)	**PASS**	Seeded mock data includes both stale cases; cart page visually distinguishes them
Testability for grading	**GOOD**	Explicit dev-only "Test Control Center" panel lets a grader trigger 400/404 without backend access

Issues & Weaknesses Found

- **Guest flow / hardcoded category:** handleAddToCart always sets category_id: 'cat_neon' regardless of the actual product being configured — fine for the current single-product mock, but not generalized.

- **Race condition risk in resume flow:** the login page uses its own 1000ms setTimeout redirect to /cart, while AuthContext separately auto-fires the resume add-to-cart on the user state change. Both are timing-based rather than sequenced/awaited — works in the mock, but is fragile under slower network/AI conditions (could briefly show an empty cart before the item appears).
- **No client-side floor enforcement beyond the UI:** server validates quantity <= 0, but this is only defense-in-depth since the quantity stepper already prevents going below 1 — not a real gap.
- **No real backend / persistence:** cart state lives in an in-memory global.mockCartStore inside a Next.js API route. This resets on every server restart and does not separate carts per logged-in user — acceptable for a frontend-only assignment per the brief, but worth flagging.
- **Auth is fully mocked:** login() accepts any email with no password check and no real backend — consistent with a frontend-focused module scope.
- **Could not verify compilation/runtime** due to the sandboxed, no-network review environment. This is a static review only; it does not confirm zero TypeScript/ESLint errors or that the pinned Next.js 16 + React 19 + Tailwind 4 versions in package.json actually resolve without conflicts.
- **Dev-only test panel:** the "Test Control Center" is correctly gated behind process.env.NODE_ENV === 'development', so it will not leak into a production build — but a grader building in production mode won't see it and will need another way to trigger 400/404 states.

Overall Score

85 / 100

Solid implementation of all core deliverables

The core deliverables from the assignment — Add to Cart wiring, guest→auth resume flow, inline 400/404 error handling, stale-item states, and AI preview hand-off — are all correctly and thoughtfully implemented, including a deliberate self-test harness for grading. Points were withheld for: inability to verify actual build/runtime success in this sandboxed environment, the somewhat fragile double-timer sequencing in the resume flow, and the fully-mocked (non-persistent, non-multi-user) backend layer, which is acceptable for a frontend module but not production-grade.

Recommendation

Run **npm install && npm run build && npm run dev** locally (with network access) to confirm zero compile errors, then manually walk through: (1) add-to-cart as a guest → login → verify auto-resume; (2) toggle the 404 test switch; (3) select the "Deprecated Font" option to trigger 400 inline errors; (4) confirm the cart page renders both stale-item states correctly.